

Zupurb — Partner Roles & Interfaces Plan

Status: planning. Owner: Alan. Created during role-architecture session. **Decisions locked:** (1) **Separate app per role** (not a unified portal). (2) Build **all roles, one by one**.

1. The role model (grounded in existing app entities)

The user app already models the full supply side; only **Venue owners** have a management interface today.

Entity in app	Role	Interface today	Plan
Establishment	Venue / restaurant owner	✅ zupurb_owner (Next.js)	Keep; template for the rest
Entertainer (DJ/band/comedian)	Entertainer	❌ view + <i>mock</i> booking	New app zupurb_entertainer
Vendor (caterer/photographer/event svc)	Service vendor	❌ view only	New app zupurb_vendor
Brand (sponsor/product, non-local)	Brand partner	❌ view only	New app zupurb_brand
Post.persona = Influencer/Artist, "Creators" tab	Influencer / creator	❌ concept only	In-app creator tools (recommended) or zupurb_creator
custom claims admin / staff	Admin / staff	Retool / Admin SDK	Optional internal console later

2. Architecture — separate app per role (per decision)

- Each business role ships as its **own Next.js web app**, scaffolded by **copying** zupurb_owner (auth context, dashboard layout, Firebase init, component kit).
- All apps share ONE Firebase project + ONE backend** (Firestore, callables, rules). Only the frontends are separate.
- Mitigate duplication:** extract the shared pieces of zupurb_owner (auth context, layout shell, Firebase client, UI components) into a shared template/package so each new app doesn't re-implement login/role-gating. Do this when scaffolding app #2.
- Creators:** recommended to live **in the Flutter app** (influencers are mobile consumers, not desk users). A separate zupurb_creator web app is possible but lower value — decide at Phase E.

3. Backend foundation (Phase A — build FIRST, shared by all apps)

Generalize the existing owner mechanics (`owners/{uid}` + `businessOwner` claim + `ownerUids` + `isEstablishmentOwner()` + `claimRequests` + `claimOps.ts`).

1. **Account types** — extend `accountType` in `functions/src/lib/schema.ts` to include `entertainer` , `vendor` , `brand` (already have `user` , `business`).
2. **Custom claims** — add `partner: true` + `partnerType: 'venue' | 'entertainer' | 'vendor' | 'brand'` (generalizes `businessOwner`).
3. **Partner link doc** — `partners/{uid}` = { `partnerType`, `managedIds: string[]`, `verified`, `createdAt` } (generalizes `owners/{uid}`). One per partner account; `managedIds` points at the entity doc(s) they control.
4. **Claim / verification flow** — extend `claimRequests` with `entityType` so one admin-approval path (`claimOps.ts`) grants the right claim + writes `partners/{uid}` .
5. **Rules helpers** — add `isPartner()` and `isPartnerOf(entityType, entityId)` to `firestore.rules` ; gate `entertainers/{id}` , `vendors/{id}` , `brands/{id}` updates on it (mirror `isEstablishmentOwner`).
6. **Bookings persistence** — `bookings/{id}` = { `entertainerId`, `userId`, `status`, `date`, `partySize`, `message`, `quote`, `createdAt` } ; callables `createBooking` (replaces the mock in `booking_sheet.dart`) and `respondToBooking` (entertainer app). **Prerequisite for the entertainer dashboard to have anything to manage.**

⚠ All of this must be **deployed to** `zupurb-9580f` — still blocked on partner/owner deploy access (the account-switch step).

4. Per-role app feature sets (MVP → later)



`zupurb_entertainer` — build first (booking UI already exists, just mock)

- **MVP:** profile editor (role, genres, tagline, photos, city), **availability calendar**, **booking/inquiry inbox** (accept / decline / quote), messaging (`entertainer_user` chat type already in schema), reviews + respond, basic stats.
- **Later:** media/EPK gallery, promoted listings, payouts.



`zupurb_vendor`

- **MVP:** profile + category, **service packages & pricing**, inquiry inbox, quotes, reviews + respond.
- **Later:** portfolio gallery, calendar, deposits.



`zupurb_brand`

- **MVP:** brand profile, **run offers/deals** (reuse the deals engine), sponsored placement in Explore.
- **Later:** campaign analytics, redemption tracking, creator-collab briefs.

★ Creator program (in-app, recommended)

- **MVP:** Creator Program enrollment + verified badge, **post/review analytics** (views, saves, follower growth), disclosure tools (disclosure screen exists).
- **Later:** brand↔creator collab marketplace, perks/payouts, promoted posts.

5. Build order (one by one)

Step	Deliverable
A	Backend foundation (\$3) — schema, claims, <code>partners</code> , claim flow, rules, bookings
B	<code>zupurb_entertainer</code> app + real bookings (scaffold from <code>zupurb_owner</code>)
C	Extract shared template/package while building app #2 = <code>zupurb_vendor</code>
D	<code>zupurb_brand</code> + offers
E	Creator program (in-app)
F	Admin console (optional)

6. Immediate next steps

1. ☒ Architecture decided: separate apps, all roles, sequential.
2. Resolve deploy access to `zupurb-9580f` (or repoint to a project you own).
3. Implement Phase A backend (\$3) and deploy.
4. Make entertainer booking persist (turns the existing user-app UI real).
5. Scaffold `zupurb_entertainer` from a copy of `zupurb_owner` .

Open questions

- Creators: in-app (recommended) vs separate `zupurb_creator` web app?
- One generalized `partners/{uid}` collection vs per-role collections (`entertainers/{uid}` admins, etc.)? (Plan assumes generalized.)
- Where does this sit relative to the `CLAUDE.md` Phase 1A→1B mockup track? (This is net-new partner-side work.)